

Conditional compilation

Unit 2.4

Why do we need conditional compilation?

- The previous example (`rtl.c`) made calls to the OpenMP runtime library
- If we don't enable OpenMP with the appropriate compiler flag then the program will not compile
- This is unsatisfactory! We want to be able to build the serial version of the program

```
/tmp/ccM6zCTE.o: In function `main':  
rtl.c:(.text+0xa): undefined reference to `omp_get_num_procs'  
rtl.c:(.text+0x20): undefined reference to `omp_get_num_threads'  
rtl.c:(.text+0x27): undefined reference to `omp_get_thread_num'  
rtl.c:(.text+0x3f): undefined reference to `omp_get_num_procs'  
rtl.c:(.text+0x55): undefined reference to `omp_get_num_threads'  
rtl.c:(.text+0x5c): undefined reference to `omp_get_thread_num'  
collect2: error: ld returned 1 exit status
```

C pre-processor

- The C pre-processor processes source code before it is passed to the compiler
- We used the pre-processor in previous examples to include files (`#include`)
- We can also use the pre-processor to include or exclude calls to OpenMP functions as required

C pre-processor: `#ifdef`

- We use the `#ifdef` pre-processor directive for conditional compilation:

```
#ifdef MACRO
```

```
    included if MACRO is defined
```

```
#else
```

```
    include if MACRO is not defined
```

```
#endif
```

- The OpenMP standard specifies that the `_OPENMP` macro is defined (if OpenMP is enabled and pre-processing is supported)

```
#include <stdio.h>
#ifdef _OPENMP
#include <omp.h>
#endif

int main () {

#pragma omp parallel
{
#ifdef _OPENMP
printf("Using OpenMP. I am thread %d\n", omp_get_thread_num());
#else
printf("Not using OpenMP\n");
#endif
}

return 0;
}
```

Unit summary

- When OpenMP is enabled the `_OPENMP` pre-processor key is automatically set
- We can use this key to exclude OpenMP library calls when OpenMP is not being used
- This enables us to compile and run the serial version of the program even if it includes calls to the OpenMP runtime library